

Design and Implementation of Efficient Stochastic Number Generator

Sanjith Reddy P V, Sai Sanath Potnuru, T Sai Venkatesh, Sonali Agrawal

Department of Electronics & Communication Engineering

Amrita School of Engineering, Bengaluru

Amrita Vishwa Vidyapeetham, India

Abstract—Stochastic Computing (SC) is a logical computational technique that performs operations on random bit streams. These random bit streams are generated using stochastic number generators (SNG). In the circuits using stochastic computing, SNGs consume most of the physical area and power. This paper proposes power and speed efficient 4-bit and 8-bit SNGs by using modified Non-linear Feedback Shift Register (NLFSR) as Random Number Source (RNS) and by applying the concept of sharing of an RNS with many SNGs. An n bit NLFSR can generate all 2^n random sequences. The 4-bit and 8-bit NLFSRs, designed using the bipartite technique, are proposed to reduce the delay and power consumed by the SNG. In this paper, proposed 4-bit and 8-bit SNGs are designed and implemented using Xilinx ISE tool and their performance concerning area, power, and delay, are evaluated with Cadence Genus synthesis solution in 45nm technology. The performance of proposed SNGs is compared with the existing SNGs and this comparison shows that the proposed SNGs are more efficient in terms of power and speed. It is also observed that as the number of bits increases, the reduction in power consumption and delay also increases for the proposed design with respect to the existing design.

Keywords— Soft computing, stochastic number generator, non-linear feedback shift register, random number source, probability conversion circuit.

I. INTRODUCTION

Stochastic computing (SC) is coming to the limelight in recent years as a new, unconventional computation technique that processes data in the form of random bit streams called stochastic numbers. The conventional deterministic binary computation technique is used to perform operations on binary numbers whereas the stochastic computing technique is used on random bit streams. Stochastic computing helps in the implementation of designs with low cost and is thus used in various fields such as neural networks [1], spectral transforms, digital filters [2], image processing [3], and cryptography applications [4], [5]. In the modern world, cryptography [6] is becoming cardinal as hackers are increasing and data must be protected [7]. Input and output are encoded as probabilities of 1's in the corresponding bit streams. The main advantage of stochastic computing is a reduction in hardware complexity and high fault tolerance during computing.

In SC, the random bit streams to be computed are generally generated using a stochastic number generator (SNG). An SNG has two parts namely; Random Number Source (RNS) and Probability Conversion Circuit (PCC). The RNS generates random bit streams. Mostly, Linear Feedback Shift Registers (LFSRs) are used as RNS. An n -bit LFSR can generate 2^n-1 random sequences only.

Nowadays, to achieve all 2^n random sequences from an n -bit RNS, non-linear LFSRs (NLFSRs) are becoming popular.

The PCC, as the name suggests, converts the bit streams into a stream having desired probability. The PCC converts each combination of bit stream generated by RNS into a single bit such that the probability of 1's in the final bit stream is equal to the probability of 1's in the deterministic binary input given to the PCC. In the SC circuits, the SNG plays the most important role. It also contributes to most of the space taken to implement the circuit. This becomes a problem in SC circuits that have many SNGs as these SNGs take up nearly 80% of the circuit area [8]. This means that to reduce the area of the SC circuits, the area of the SNGs is to be reduced to ensure a significant change in the area. This applies to the power consumed by the circuit too. When SNGs of higher order are considered, the amount of power taken to drive the circuit is high.

This paper deals with the challenge of reducing the amount of power consumed by the SNG used in an SC circuit. In response to this challenge, the contributions of this paper are as follows-

- 1) This paper is proposing 4-bit and 8-bit power and speed efficient NLFSRs. These NLFSRs are used as RNS in the SNG to reduce the amount of power consumed by the SNG and delay which in turn reducing the power consumption of the SC circuit and improving its speed.
- 2) The sharing of RNS among many SNGs also reduces the area and power consumed by the SC. The sharing of RNS is applied in proposed SNG models.

The rest of the sections in the paper are arranged as follows; Section II provides the preliminaries and prior work about the SNG. Section III explains the proposed RNS and proposed SNGs. Section IV shows the performance analysis and comparison result of the proposed model with the existing SNG model. The conclusion of the work is described in Section V.

II. PRELIMINARIES AND PRIOR WORK

A. Stochastic Number Generator and SCC

Stochastic Number Generator (SNG) is the basic part of Stochastic Computing. It generates random bit streams of desired probabilities of 1's. Fig. 1 shows the general structure of an SNG where LFSR is used as RNS. Generally, an SNG consists of two parts, the random number source (RNS) and the probability conversion circuit (PCC). There are two types of RNS namely the cellular automata (CA)

and the Feedback shift registers (FSRs). A CA consists of serially connected modules called a cell or site. Each cell is composed of a flip-flop and a combinational circuit. Although a CA can generate good-quality random numbers, it is rarely used in SC circuits as it has a high hardware complexity compared to an FSR. Due to low complexity and high speed, FSRs are preferred in SC circuits over CA.

The FSR consists of cascaded flip-flops along with a feedback path to the first flip-flop. The feedback path can be written as an equation and based on this equation, the FSRs are of two types namely the Linear Feedback Shift Register (LFSR) [9] and the Non-Linear Feedback Shift Register (NLFSR) [10]. The LFSRs consist of a feedback path which can be described using a linear function whereas the NLFSRs have a feedback path that has a non-linear function.

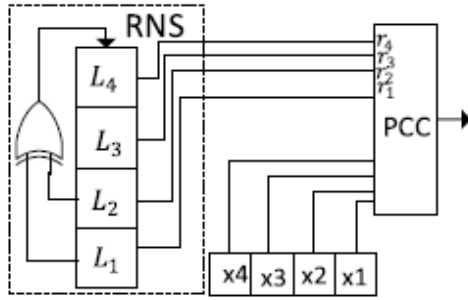


Fig. 1. Basic Structure of an SNG using LFSR

Fig. 2 shows an example of an NLFSR. The NLFSR has nonlinear feedback as there is an AND gate which increases the order of the feedback function [11].

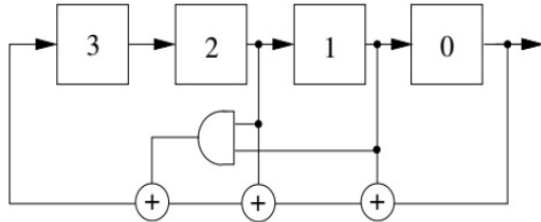


Fig. 2. Nonlinear Feedback Shift Register

Generally, an LFSR generates 2^n-1 sequences which means that the output succession repeats after 2^n-1 combinations of its output and doesn't have a combination having all 0s. So, if LFSR is used in the SNG, then 2^n-1 bits are produced. On the other hand, an NLFSR produces all combinations including all 0s, and thus produces 2^n combinations in succession before the combinations get repeated and when used in the SNG the output has 2^n bits.

Probability Conversion Circuit (PCC), as the name suggests, gives the output bit streams which has a desired probability as per the user requirement. The PCC works by taking the input from the RNS and converts it into a random bit stream with the probability of 1's equal to the probability of 1's in the given deterministic binary input bit stream. In other words, the desired probability is converted into bit streams. It has two n-bit inputs, one from the RNS and other a pre-determined user input, and one output. There are two types of PCC. They are named as Weighted Binary Generator (WBG) and Comparator (CMP). The comparator produces 1 if binary input is greater than or equal to the

LFSR's output and 0 otherwise. While WBG produces output same as the bit in binary input whose position is equal to the leading 1's position in input from LFSR and 0 if input from LFSR is 0 [12]. The CMP PCC is shown in Fig. 3(a) and WBG PCC is shown in Fig. 3(b). Due to the type of logic gates used in WBG, WBG consumes less power and requires less area as compared to CMP.

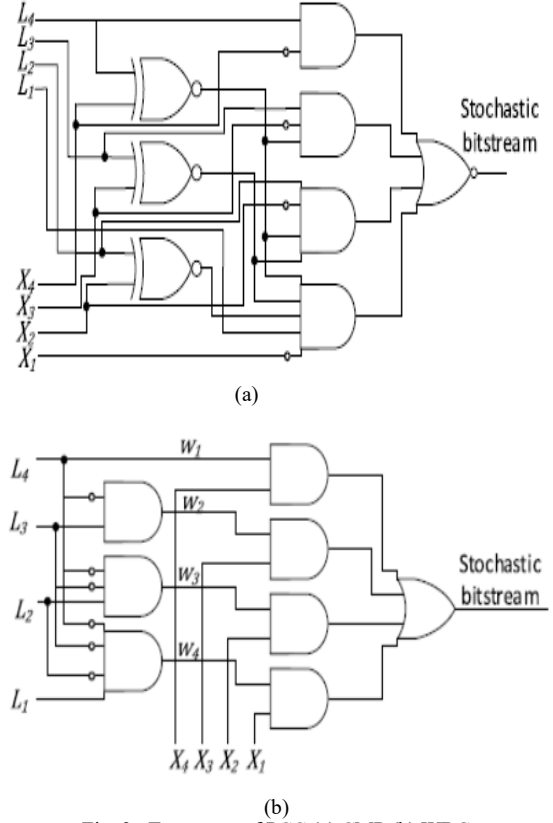


Fig. 3. Two types of PCC (a) CMP (b) WBG

Stochastic Computing Correlation (SCC) is the measure of the correlation between two-bit streams [13]. If the correlation between two-bit streams generated by SNG is more, then the accuracy of the computation is affected. Let s_x and s_y be the random bit streams generated by binary numbers x and y respectively, then SCC is given by (1)

$$SCC(s_x, s_y) = \begin{cases} \frac{\partial(s_x, s_y)}{\min(P(s_x), P(s_y)) - P(s_x)P(s_y)} & \text{if } \partial(s_x, s_y) \geq 0 \\ \frac{\partial(s_x, s_y)}{P(s_x)P(s_y) - \max(P(s_x) + P(s_y) - 1, 0)} & \text{otherwise} \end{cases} \quad (1)$$

where $P(s_x)$ and $P(s_y)$ are probabilities of 1's in bit stream s_x and s_y and $\partial(s_x, s_y) = P(s_x \wedge s_y) - P(s_x)P(s_y)$ where $\wedge$ denotes bitwise AND operation of s_x and s_y . SCC varies between -1 and +1. A value of 0 is desired as SCC which means that there is no correlation. If all corresponding bits of two-bit streams are equal, then SCC is 1 and it becomes -1 if they are complemented. The average SCC between two SNGs is calculated by

$$SCC_{avg}(SNG_1, SNG_2) = \sum_{i=0}^{2^n-1} \sum_{j=0}^{2^n-1} \frac{|SCC(s_i, s'_j)|}{2^n \times 2^n} \quad (2)$$

where s_i, s'_j represent the bit streams generated by SNG1 and SNG2 respectively and all possible inputs that can be given to both SNGs in all combinations possible.

B. Prior Work

In many applications, more than one bit stream is required in the SC circuit. It is implemented directly using a different SNG for generating each bit stream. Instead of direct implementation, sharing of RNS with many SNG reduces the cost in terms of area and power. Direct sharing of SNGs leads to the high correlation between the outputs which leads to inaccurate computations. So Neugebauer et al. [14] recommended the use of an extra S box while sharing LFSRs to reduce the correlation. But in this proposal, there are chances of increase in hardware complexity because of extra circuits as n increases. Another method of circular shifting [15] is done before connecting PCCs with the shared RNS which circularly shifts the combinations by k values. But the reduction in correlation is not enough. Sayed Ahmed Salehi [16] proposed sharing in terms of permutations where each combination is considered as a permutation and an algorithm created that gives the permutation where SCC is minimum. The paper also suggests that WBG is cost-efficient compared to a CMP as PCC in terms of area and power.

In all the above works, there are unnecessary transitions happening in the LFSR (which is used as RNS) which if reduced, the cost in terms of power can be reduced. The proposed model uses NLFSR as RNS and another modification is done in NLFSR to reduce the switching activity of flip flops and thus reduce the dynamic power. As it is already mentioned that WBG is better than CMP in terms of area and power, the WBG is used as PCC in the proposed model.

III. PROPOSED MODEL

This section explains about the proposed model for RNS and SNG. The modified RNS is explained for both 4-bit and 8-bit size and the proposed SNGs are designed and implemented using these modified RNS.

A. Proposed RNS

LFSR is a feedback shift register that generates $2^n - 1$ combinations and doesn't generate all zero combination at the output. Because of this, there are only $2^n - 1$ bits available at the output of the SNG. On the other hand, an NLFSR generates all 2^n combinations including all zeroes and thus all output bits are available at the output of SNG. Fig. 4 shows a 4-bit NLFSR that is used in the SNG to get all possible output bits. It has a feedback function of $Y = x_3 + x_0 + x_3'x_2'x_1'$ where $+$ denotes XOR operation. As the equation is of order three, it is non-linear in nature. The number of switching activities that happened in the NLFSR while generating random sequences is 30 whereas it is 32 in an LFSR. Because of the extra circuitry in NLFSR compared to LFSR, the cost of the circuit increases in terms of power, area, and delay. In the NLFSR, the number of switching activities can be reduced by using the bipartite technique [17].

In the bipartite technique, an N -bit FSR is divided into two parts each of size $N/2$ as shown in Fig. 5. The first part

works at every clock cycle while the second part works only in a scenario based on the first part's output. The second part does not work for each clock cycle which reduces the number of switching activities in the generated random sequences.

Fig. 6 shows the implementation of the bipartite technique in 4-bit NLFSR. In this modified NLFSR, four flip-flops are divided into two parts. Each part internally has nonlinear feedback and thus maintaining non-linearity internally. The first part receives normal external clock

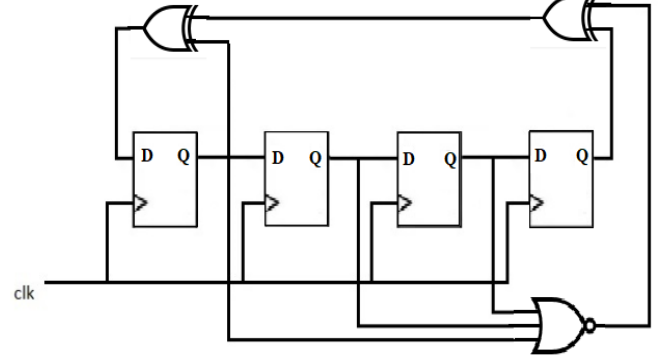


Fig. 4. Normal NLFSR

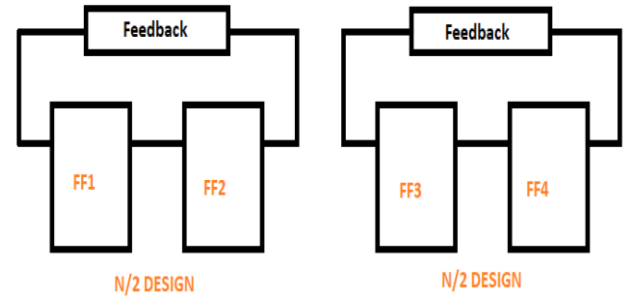


Fig. 5. Bipartite Technique

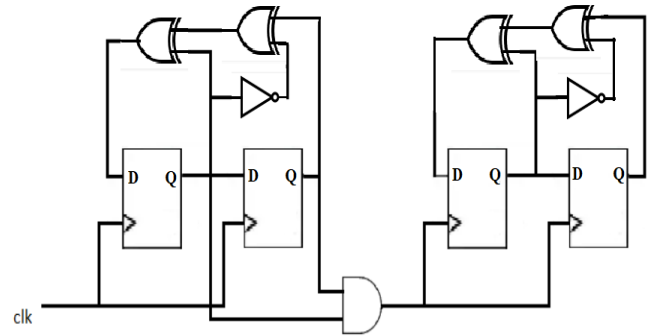


Fig. 6. Modified 4-bit NLFSR

while the second part's clock is the output of AND operation of outputs of the first two flip flops. In other words, last two flip-flops work only when the first two flip-flops output is 1. By doing so the number of switching activities, in the generation of all the random sequences, are reduced. Table I shows the number of switching activities that happen in different types of 4-bits FSRs. Because of the division of the FSR into two parts, the number of switching activities decreases from 30 in normal NLFSR to 18 in modified NLFSR. It can be seen that the last two bits are changed non-linearly only when the first two bits are 1s.

TABLE I
SWITCHING ACTIVITY IN DIFFERENT MODELS OF 4-BIT FSRS

Clock	LFSR	NLFSR	Modified NLFSR
1	1111	1111	1111
2	0111	0111	0111
3	1011	1011	0011
4	0101	0101	1011
5	1010	1010	1101
6	1101	1101	0101
7	0110	0110	0001
8	0011	0011	1001
9	1001	1001	1100
10	0100	0100	0100
11	0010	0010	0000
12	0001	0001	1000
13	1000	0000	1110
14	1100	1000	0110
15	1110	1100	0010
16	1111	1110	1010
Switching	8-8-8-8	8-7-8-7	8-7-2-1
Total	32	30	18

Fig. 7 shows 8-bit Modified NLFSR. Eight flip flops are divided into two 4-bit modified NLFSR. The second part's clock is connected to AND of the first part's output. The second part will work only when the outputs of the first four flip flops are 1. Internally the last two flip flops run when outputs of the fifth and sixth flip-flops are 1 and also when the first part is active i.e., when the first six flip flop's output is 1. This implementation reduces the switching even more than the reduction in 4-bit modified NLFSR.

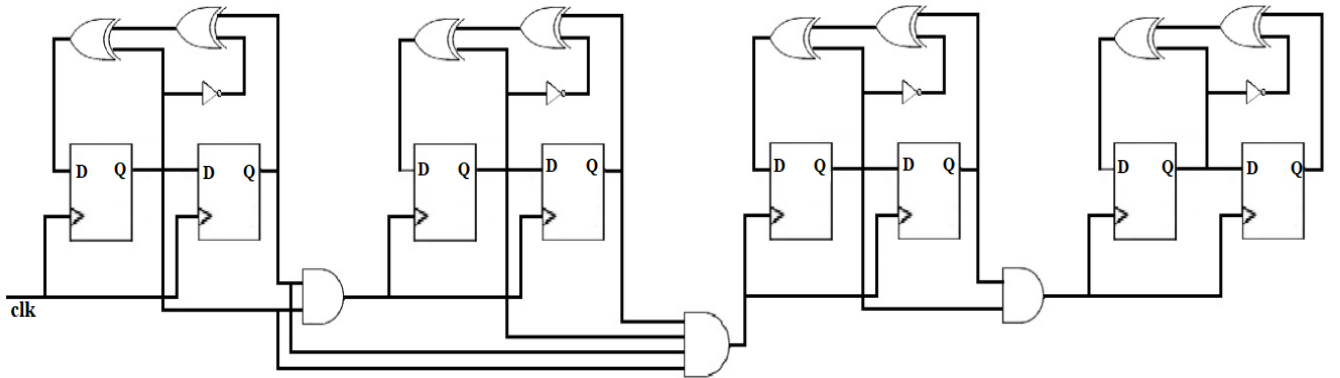


Fig. 7. Modified 8-bit NLFSR

B. Proposed SNG

An SNG can be implemented using the proposed NLFSR and PCC. Weighted Binary Generator (WBG) is more efficient compared to CMP [16] in terms of SCC, area, and power. So WBG is the PCC used in the proposed model of SNG.

Fig. 3(b) shows a 4-bit WBG and Fig. 8 shows the 8-bit implementation. In a WBG, the LFSR output is transformed into weighted binary numbers and then outputs are generated using deterministic binary input and the previously generated weighted binary numbers. In applications where many random bits streams are required, sharing of RNS i.e., modified NLFSR, can be done by permutation technique [16].

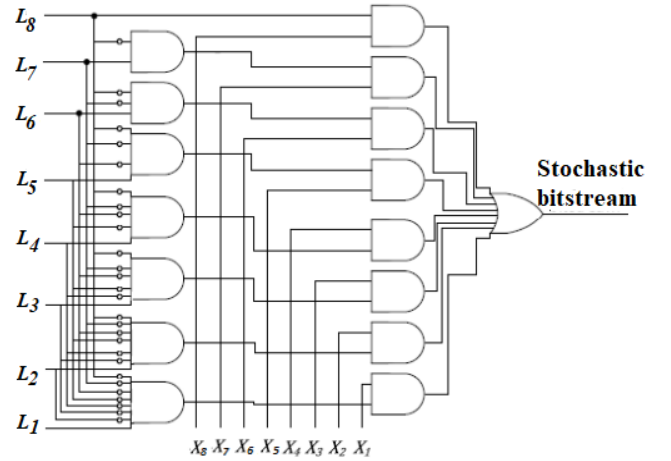


Fig. 8. 8-bit WBG

The output of the modified NLFSR is considered in reverse lexicographical order of permutations. In reverse lexicographical order, the vector permutations are done on the index of the position of the elements i.e., vectors are obtained by reordering the elements from left to right starting from the largest index [16]. Then the best m permutation that has the least SCC_{avg} value is found out using Algorithm 1 [16], where m is the number of WBGs with which the RNS is shared.

Algorithm 1 Algorithm for Finding the Set of m Permutations of an LFSR With Minimum Pairwise SCC_{avg}

```

Input:  $m$ 
Outputs:  $P_1, P_2, \dots, P_m$ 
Initialization:  $SM=1$ 
for  $i = m$  to  $n!$  do
  for  $j = 1$  to  $i - (m - 1)$  do
    if  $SCC_{avg}(PL_i, PL_j) < SM$  then
       $SP_1 = SCC_{avg}(PL_i, PL_j)$ 
      for each set  $(k_1, k_2, \dots, k_{m-2})$  of  $m - 2$ 
        elements in  $\{j + 1, j + 2, \dots, i - 1\}$  do
          if  $SCC_{avg}(PL_i, PL_{k_r}) < SM$  for all  $k_r \in (k_1, k_2, \dots, k_{m-2})$  then
            for all  $k_r$  do
               $SP_{k_r} = SCC_{avg}(PL_i, PL_{k_r})$ 
              if  $SCC_{avg}(PL_{h_1}, PL_{h_2}) < SM$  for all pairs  $(h_1, h_2)$ 
                in  $(j, k_1, k_2, \dots, k_{m-2})$  then
                   $SP_{h_1 h_2} = SCC_{avg}(PL_{h_1}, PL_{h_2})$ 
                   $P_1 = i$ 
                   $P_2 = j$ 
                   $P_3 = k_1$ 
                   $\vdots$ 
                   $P_m = k_{m-2}$ 
             $SM = \max \{all\ SP_q s: SP_1, SP_{k_1}, \dots, SP_{k_{m-2}}, SP_{h_1 h_2}, \dots\}$ 

```

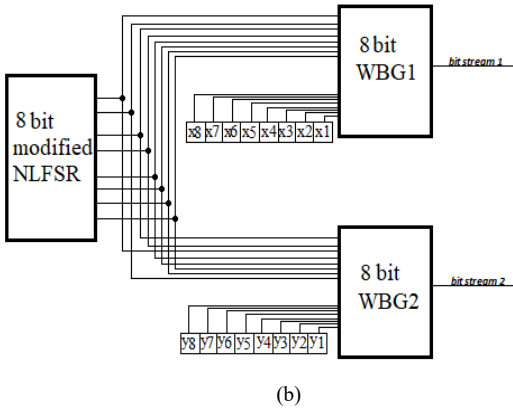
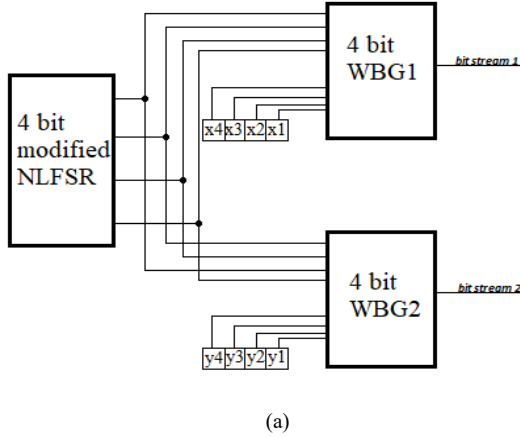


Fig. 9. Sharing of RNS in SNG (a) 4-bit (b) 8-bit

Fig. 9(a) shows the implementation of 4-bit SNG with one shared proposed 4-bit modified NLFSR with two WBGs and Fig. 9(b) shows the sharing of an 8-bit modified NLFSR with two WBGs to design the proposed 8-bit SNG.

IV. RESULTS AND PERFORMANCE ANALYSIS

All RNS, PCC, and SNG circuits are coded using Verilog HDL and simulated with Xilinx ISE tool. Cadence Genus synthesis solution with 45nm technology is used for performance evaluation and comparison, concerning power, area, and delay, between existing and proposed models of RNS and SNG.

Table II shows the performance comparison among LFSR, NLFSR, and modified NLFSR. In NLFSR all parameters are increased as compared to normal LFSR. So, NLFSR is not a better model than LFSR even though it gives all 16 combinations in its output including 0000. But when this NLFSR is modified with the help of the bipartite technique, it is giving improved results.

There is a 33.7% decrease in power and a 9.79% reduction in delay with a trade-off of 0.91% increase in area in modified NLFSR as compared to normal LFSR. This model also produces all 16 combinations of output including 0000. With the decrease in power in modified NLFSR, it is evident that the reduction in switching activities among the FSR can reduce power and making it power efficient.

TABLE II
PERFORMANCE COMPARISON OF 4-BIT RNS

4-bit RNS	AREA (μm^2)	POWER (nW)	DELAY (ps)
LFSR [13]	122.63	21265.723	490
NLFSR	125.804	21882.546	586
% Change in NLFSR w.r.t Normal LFSR	2.58% Increased	2.9% Increased	19.59% Increased
Modified NLFSR	123.74	14089.15	442
% Change in Modified NLFSR w.r.t LFSR	0.91% Increased	33.7% Decreased	9.79% Decreased

Table III shows the performance comparison between 8-bit LFSR and 8-bit Modified NLFSR. As there is more decrease in switching activities in 8-bit Modified NLFSR, the improvement is also more when compared with 4-bit. There is a decrease of 51.8% in power and 25.96% in delay in modified 8-bit NLFSR as compared to the existing normal 8-bit LFSR given in [16].

Though there is a 2.8% increase in area, it can be concluded that 8-bit NLFSR is efficient compared to 8-bit LFSR [16]. Also, 8-bit LFSR generated 255 combinations, while modified NLFSR generates 266 combinations including 00000000.

When sharing of a 4-bit modified RNS is done with two 4-bit WBGs to generate two bit streams in the proposed SNG, the performance analysis is shown in Table IV. In the modified NLFSR there is a reduction of 24.8% in power and 9.56% in delay. In the case of modified NLFSR, the output bit streams generate 16 different combinations before they get repeated, unlike LFSR where the bit streams repeat after 15 clock cycles. So, SNG with modified NLFSR has less power and less delay.

TABLE III
PERFORMANCE COMPARISON OF 8-BIT RNS

8-bit RNS	AREA (μm^2)	POWER (nW)	DELAY (ps)
LFSR [16]	248.03	42794.41	597
Modified NLFSR	255.77	20622.89	442
% Change in modified NLFSR w.r.t NFSR	2.83% Increased	51.81% Decreased	25.96% Decreased

TABLE IV
PERFORMANCE COMPARISON OF SNG WITH SHARING OF A 4-BIT RNS WITH TWO 4-BIT WBGs

SNG with Shared RNS Model	AREA (μm^2)	POWER (nW)	DELAY (ps)
SNG with LFSR	218.014	36229.433	533
SNG with Modified NLFSR	219.857	27226.997	482
% Change in SNG with NLFSR as compared to SNG with LFSR	0.967% Increased	24.8% Decreased	9.56% Decreased

Table V shows performance comparison when 8-bit RNS is shared with two 8-bit WBGs with the permutation vector obtained from algorithm 1. The power is reduced by 48.66% and delay is reduced by 23.77% in the proposed SNG, obtained by sharing modified NLFSR with two 8-bit WBGs, compared to existing SNG designed with normal 8-bit LFSR. The percentage of reduction in power is more in the 8-bit model because the number of switching activities that reduced in modified NLFSR is more in 8-bit compared to 4-bit proposed model. Even though the area increased by 3%, the new proposed model for 8-bit is efficient than the existing LFSR when SNG is implemented. So, the SNG designed using modified RNS with the concept of sharing of RNS among many SNGs provides power and speed efficient stochastic number generator. This SNG can be used in various applications like image processing, neural network, cryptography, digital filters, spectral transforms etc.

TABLE V
PERFORMANCE COMPARISON OF SNG WITH SHARING OF A 8-BIT RNS WITH TWO 8-BIT WBGs

SNG with Shared RNS Model	AREA (μm^2)	POWER (nW)	DELAY (ps)
SNG with LFSR	443.84	81897.83	631
SNG with Modified NLFSR	457.5	42043.52	481
% Change in SNG with NLFSR as compared to SNG with LFSR	3.078% Increased	48.66% Decreased	23.77% Decreased

V. CONCLUSION

In this paper, power-efficient and fast 4-bit and 8-bit RNS models have been proposed which are used in the basic element of Stochastic Computing i.e., Stochastic Number Generator (SNG). These proposed RNS models are modified NLSFRs in which the bipartite technique is used. In these modified NLFSRs, the power and delay are reduced due to reduction in the number of switching activities while generating random sequences. These modified RNS models (modified NLFSR) when compared to existing RNS models (LFSR), there is a decrease of 33.7% in power, 9.7% in delay for 4-bit model, and 51.81% decrease in power, 25.96% decrease in delay for the 8-bit model. These RNS models when used to design proposed SNGs for 4-bit and 8-bit size where an RNS is shared with two WBGs to generate two-bit streams, there is a decrease of 24.8% in power, 9.56% decrease in delay in the 4-bit SNG model and 48.66% of reduction in power and 23.77% decrease in delay in the 8-bit SNG model. It is also observed that as the number of bits n increases in the proposed RNS and SNG model, the reduction in switching activities also increases which reduces the power and delay by more amount. This concluded that the proposed SNG models are power and speed efficient and as the number of bits n increases the amount of power and delay reduction also increases in these models. Further, WBG can also be made efficient by working on gate level improvement of the circuit. Galois method can be implemented in NLFSR model and proposed model thus can reduce the cost even further.

REFERENCES

- [1] B. Brown and H. Card, "Stochastic neural computation. I. Computational elements," IEEE Transaction on Computers., vol. 50, no. 9, pp. 891–905, Sep. 2001.
- [2] H. Ichihara, T. Sugino, S. Ishii, T. Iwagaki, and T. Inoue, "Compact and accurate digital filters based on stochastic computing," IEEE Transaction on Emerging Topics in Circuits and Systems, vol. 7, no. 1, pp. 31–43, Jan. 2019.
- [3] S. R. Prasad, Siripagada, A., Selvaraj, S., and N Mohankumar, "Random seeding LFSR-based TRNG for hardware security applications", Studies in Computational Intelligence, vol. 771, pp. 427-434, 2019.
- [4] P. Li, D. J. Lilja, W. Qian, K. Bazargan, and M. D. Riedel, "Computation on stochastic bit streams digital image processing case studies," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 22, no. 3, pp. 449–462, Mar. 2014.
- [5] A. Siripagada, R. Prasad, S., and Mohankumar, N., "Power efficient PUF-based random reseeding true random number generator", Advances in Intelligent Systems and Computing, vol. 900, pp. 549-559, 2019.
- [6] P. P. Mahapatra and S. Agrawal, "RSA Cryptosystem with Modified Montgomery Modular Multiplier," IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), pp. 1-6, 2017.
- [7] R. S. Durga, C. K. Rashmika, O. N. V. Madhumitha, D. G. Suvetha, B. Tanmai and N. Mohankumar, "Design and Synthesis of LFSR based Random Number Generator," *Third International Conference on Smart Systems and Inventive Technology (ICSSIT)*, pp. 438-442, 2020.
- [8] S. Mohajer, Z. Wang, K. Bazargan, M. Riedel, D. Lilja, and S. Faraji, "Parallel computing using stochastic circuits and deterministic shuffling networks," U.S. Patent Appl. 16/165 713, Apr. 2019.

- [9] N. Haridas and Dr. Nirmala Devi M., "Efficient linear feedback shift register design for pseudo exhaustive test generation in BIST," 3rd International Conference on Electronics Computer Technology ICECT, pp. 350-354, 2011.
- [10] V. N. Teja and Prabhu E., "Test pattern generation using NLFSR for detecting single stuck-at faults," IEEE International Conference on Communication and Signal Processing, ICCSP, pp. 716-720, 2019.
- [11] N. Krishna, Murugappan, V., Harish, R., Midhun, M., and Prabhu E., "Design of a Novel Reversible Nlfsr," International Conference on Advances in Computing, Communications and Informatics (ICACCI), pp. 2279-2283, 2017.
- [12] P. Gupta and R. Kumaresan, "Binary multiplication with PN sequences," IEEE Trans. Acoust., Speech, Signal Process., vol. ASSP-36, no. 4, pp. 603-606, Apr. 1988.
- [13] A. Alaghi and J. P. Hayes, "Exploiting correlation in stochastic circuit design," IEEE 31st International Conference on Computer Design (ICCD), pp. 39-46, Oct. 2013.
- [14] F. Neugebauer, I. Polian, and J. P. Hayes, "Building a better random number generator for stochastic computing," Euro-micro-Conference on Digital System Design (*DSD*), pp. 1-8., Aug. 2017.
- [15] H. Ichihara, S. Ishii, D. Sunamori, T. Iwagaki, and T. Inoue, "Compact and accurate stochastic circuits with shared random number sources," IEEE 32nd International Conference on Computer Design (ICCD), pp. 361-366, Oct. 2014.
- [16] Sayed Ahmad Salehi, "Low-Cost Stochastic Number Generators for Stochastic Computing," IEEE Transactions on Very Large-Scale Integration (VLSI) Systems, VOL. 28, NO. 4, pp. 992 - 1001, April 2020.
- [17] M. Tehranipoor, M. Nourani and N. Ahmed, "Low Transition LFSR for BIST-Based Applications," 14th Asian Test Symposium (ATS'05), pp. 138-143, 2005.